# SIMULATION AND COMPARATIVE ANALYSIS OF PAGE REPLACEMENT ALGORITHMS

Operating Systems — Week 5 PBL Report

| | |
|---|---|
| **Team Members** | Rachel<br>Trisha<br>Anush<br>Inchara |
| **Course** | Operating Systems, Week 5 PBL |
| **GitHub Repository** | github.com/rd8820008-boop/PBL (folder: week5) |

Prepared as a written report of the Week 5 page-replacement simulator. Application screenshots have not been included in this version.

## 1. Problem Statement

The objective of this project is to design and implement a simulator that demonstrates the working of the FIFO, LRU and Optimal page-replacement algorithms. The program accepts a page-reference string and the number of frames, displays the frame contents after every page request, identifies page hits and page faults, calculates totals and ratios, and compares the algorithms to determine which one produces the fewest page faults.

## 2. Introduction

In a virtual-memory system, only some of a process's pages are kept in physical memory at a time. These pages occupy memory locations called frames. When a requested page is not present in memory, a **page fault** occurs and the page must be loaded. If all frames are occupied, the operating system must select an existing page to remove to make room. A page-replacement algorithm makes this decision. A good algorithm aims to reduce page faults and use memory efficiently.

## 3. Algorithms

**FIFO (First In, First Out):** FIFO removes the page that has been in memory the longest. It is simple to understand and implement, but it may remove a page that is still being used frequently.

**LRU (Least Recently Used):** LRU removes the page that has not been used for the longest time. It uses past access history to estimate which page is less likely to be needed soon. It often performs better than FIFO, although its performance depends on the reference string.

**Optimal:** Optimal removes the page whose next use is farthest in the future, or which will not be used again. It produces the minimum possible number of page faults for a known reference string. However, a real operating system cannot generally know future page requests, so Optimal is mainly used as a benchmark.

## 4. Implementation

The simulator is implemented in **Python**. Its graphical user interface is a web page built using HTML and JavaScript and connected to a Flask backend.

| File | Purpose |
|---|---|
| algorithms.py | Contains separate fifo(), lru() and optimal() functions, along with input validation. |
| app.py | Connects the web interface to the page-replacement algorithms through the Flask backend. |
| templates/index.html | Provides the GUI, including input controls, step tables, a summary table, a bar chart and a Random String button. |
| sweep.py | Calculates page faults for frame counts from 1 to 7. |
| test_algorithms.py | Tests the page-replacement algorithms. |

Input validation is intended to reject an empty reference string, non-numeric input and a frame count of zero or less. The application should display an appropriate message rather than attempting an invalid simulation.

## 5. Results

**Reference string:** 7012030423 **Number of frames:** 3

The following results are the values supplied for the project run. Screenshots of the input area, individual algorithm step tables, performance summary and bar chart are not included in this copy of the report.

| Algorithm | Page hits | Page faults | Hit ratio | Fault ratio |
|---|---|---|---|---|
| FIFO | 1 | 9 | 10% | 90% |
| LRU | 2 | 8 | 20% | 80% |
| Optimal | 4 | 6 | 40% | 60% |

**Hit ratio** = (Number of page hits ÷ Total number of references) × 100

**Fault ratio** = (Number of page faults ÷ Total number of references) × 100

There are 10 page references in the given string. Therefore, the hit and fault ratios are calculated using 10 as the total number of references.

## 6. Effect of the Number of Frames

The simulator also evaluates all three algorithms using between 1 and 7 frames. In general, increasing the number of available frames can reduce the number of page faults, because more pages can remain in memory. However, this is not guaranteed for every algorithm.

**Belady's anomaly:** Belady's anomaly is a situation in which increasing the number of frames causes the number of page faults to increase. It can occur with FIFO. For the reference string **123412512345**, FIFO produces 9 page faults with 3 frames and 10 page faults with 4 frames. LRU and Optimal do not exhibit Belady's anomaly.

The faults-versus-frames table from the application is not included in this version. The values should be taken directly from the simulator if the table is required for submission.

## 7. Conclusion

For the given reference string and 3 frames, Optimal produced the fewest page faults (6), followed by LRU (8) and FIFO (9). Optimal is useful as a benchmark because it gives the minimum possible number of page faults, but it is not generally practical in a real operating system because it requires knowledge of future page requests. LRU is a practical strategy based on recent usage, while FIFO is simple to implement but may perform less efficiently and can exhibit Belady's anomaly. This project demonstrates how the choice of page-replacement algorithm affects memory performance.